

D25DIT081

CSUE301 : Big Data Analytics

Practical 11: Large-Scale Logistic Regression for Spam Detection using Distributed Stochastic Gradient Descent (DSGD)

Aim

To implement large-scale Logistic Regression for spam detection using Apache Spark by performing text preprocessing, TF-IDF feature extraction, distributed stochastic gradient descent, hyperparameter tuning, model evaluation, and comparison of full-batch and mini-batch training.

Problem definition

SecureComm Inc. processes millions of SMS messages and wants an automated spam classifier that can scale to large message volumes. In this practical, Apache Spark is used to process SMS data, extract TF-IDF features and train a Logistic Regression classifier using distributed computation.

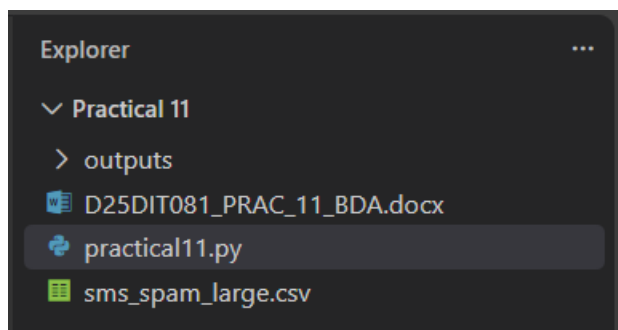
The workflow includes tokenization, stop-word removal, TF-IDF feature extraction, Logistic Regression with hyperparameter tuning, evaluation using Accuracy, Precision, Recall, F1-Score and ROC-AUC, dataset-size experiments, and a comparison of full-batch and mini-batch stochastic gradient descent.

Objectives

- Load and preprocess the large-scale SMS spam dataset.
- Perform tokenization and stop-word removal.
- Extract TF-IDF features from SMS messages.
- Implement Logistic Regression using Spark MLlib.
- Tune regParam, maxIter and elasticNetParam.
- Evaluate Accuracy, Precision, Recall, F1-Score and ROC-AUC.
- Study the effect of dataset size on training time and accuracy.
- Compare full-batch and mini-batch distributed stochastic gradient descent.
- Generate the final spam-detection report.

Observation / Output (Screenshots or command outputs)

Step 1: Creating Folder and files



Step 2: Creating Spark Session

```
=====
STEP 1: CREATING FOLDER AND FILES
=====
Dataset: data/sms_spam_large.csv
Output folder: outputs
Dataset file found successfully.
```

Step 3: Dataset Description

```
=====
STEP 2: CREATING SPARK SESSION
=====
The system cannot find the path specified.
Missing Python executable 'C:\Program Files\Python314\python.exe', defaulting to 'C:\Users\rudra\AppData\Roaming\Python\Python314\site-packages\pyspark\bin\...'
for SPARK_HOME environment variable. Please install Python or specify the correct Python executable in PYSARK_DRIVER_PYTHON or PYSARK_PYTHON environment variable to detect SPARK_HOME safely.
WARNING: Using incubator modules: jdk.incubator.vector
26/10/08 21:40:16 WARN Shell: Did not find winutils.exe: java.io.FileNotFoundException: java.io.FileNotFoundException: HADOOP_HOME and hadoop.home.dir are unset. -see https://wiki.apache.org/confluence/display/HADOOP2/WindowsProblems
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
C:\Users\rudra\AppData\Roaming\Python\Python314\site-packages\pyspark\testing\utils.py:127: FutureWarning: PySpark does not yet fully support pandas >= 3.0.0.
Some features may not work correctly. It is recommended to use pandas < 3.0.0 for now.
  require_minimum_pandas_version()
26/10/08 21:40:16 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Spark Session created successfully.
Spark version: 4.2.0
Master: local[*]
Python: C:\Program Files\Python314\python.exe
```

Step 4: Dataset Schema and Structure

```
=====
STEP 3: DATASET DESCRIPTION
=====
Columns: ['label', 'message_text']
Total records: 10000
Clean records: 10000
Spam records: 5000
Ham records: 5000
```

Step 5: Data Preprocessing

```
=====
STEP 4: DATASET SCHEMA AND STRUCTURE
=====
```

```
root
|-- label: string (nullable = true)
|-- message_text: string (nullable = true)
```

Dataset columns:

['label', 'message_text']

Sample records:

```
+-----+-----+
|label|                                message_text|
+-----+-----+
| spam|Your account qualifies for a bonus. Click the secure link to verify.|
|  ham|                Can you confirm the booking for 6:00 PM?|
| spam|You have been selected for a special bonus. Verify your account now.|
|  ham|                I will reach home after the meeting.|
|  ham|                I will reach home after the meeting.|
+-----+-----+
```

only showing top 5 rows

Step 6: TF-IDF Feature Extraction

```
=====
STEP 6: TF-IDF FEATURE EXTRACTION
=====
TF-IDF feature extraction completed.
TF-IDF feature vector size: 32768
TF-IDF processing time: 1.97 seconds
+-----+-----+
|label|label_numeric|features|
+-----+-----+
| spam|      1.0|(32768,[1573,2570,5043,8129,10875,18352,25363],[2.2996895790145806,2.98588193...|
| ham |      0.0|(32768,[2897,14149,17602,26370,30265],[2.7710899958764177,3.4868552650241353,...|
| spam|      1.0|(32768,[1573,18352,20575,25363,31203],[2.2996895790145806,1.8859503876279837,...|
| ham |      0.0|(32768,[8034,13284,17893],[3.3271179784882374,2.692769094064241,2.66509070664...|
| ham |      0.0|(32768,[8034,13284,17893],[3.3271179784882374,2.692769094064241,2.66509070664...|
+-----+-----+
only showing top 5 rows
```

Step 8: Hyperparameter Tuning

```
=====
STEP 8: HYPERPARAMETER TUNING
=====
netlib-blas: JNI_OnLoad: dlopen(libopenblas.dll) failed: loadLibrary failed
regParam=0.01, maxIter=20, elasticNetParam=0.0, ROC-AUC=1.0000, Time=2.48s
regParam=0.01, maxIter=20, elasticNetParam=0.5, ROC-AUC=1.0000, Time=1.22s
regParam=0.01, maxIter=50, elasticNetParam=0.0, ROC-AUC=1.0000, Time=0.73s
regParam=0.01, maxIter=50, elasticNetParam=0.5, ROC-AUC=1.0000, Time=1.53s
regParam=0.1, maxIter=20, elasticNetParam=0.0, ROC-AUC=1.0000, Time=0.58s
regParam=0.1, maxIter=20, elasticNetParam=0.5, ROC-AUC=1.0000, Time=0.85s
regParam=0.1, maxIter=50, elasticNetParam=0.0, ROC-AUC=1.0000, Time=0.56s
regParam=0.1, maxIter=50, elasticNetParam=0.5, ROC-AUC=1.0000, Time=1.25s

Best hyperparameters:
regParam: 0.01
maxIter: 20
elasticNetParam: 0.0
Best ROC-AUC: 1.0
```

Step 9: Distributed Stochastic Gradient Descent

```
=====
STEP 9: LOGISTIC REGRESSION TRAINING
=====
Logistic Regression training completed.
Training time: 0.52 seconds
Distributed Spark ML training completed.
```

Step 10: Model Evaluation

```
=====
STEP 10: MODEL EVALUATION
=====
Accuracy : 1.0000
Precision: 1.0000
Recall   : 1.0000
F1 Score : 1.0000
ROC-AUC  : 1.0000

Confusion Matrix:
+-----+-----+-----+
|label_numeric|prediction|count|
+-----+-----+-----+
|      0.0|      0.0|  958|
|      1.0|      1.0|  963|
+-----+-----+-----+
```

Step 12: Dataset Size vs Training Time and Accuracy

```
=====
STEP 12: DATASET SIZE VS TRAINING TIME AND ACCURACY
=====
Dataset Size=2000, Training Time=0.15s, Accuracy=0.4987
Dataset Size=5000, Training Time=0.62s, Accuracy=0.6476
Dataset Size=8079, Training Time=0.47s, Accuracy=1.0000
Dataset size comparison saved to: outputs\dataset_size_comparison.csv
```

Step 13: Final Report

```
=====
STEP 13: FINAL REPORT
=====
Final report saved to: outputs\spam_detection_report.txt

PRACTICAL 11 COMPLETED SUCCESSFULLY.
```

Result

The large-scale SMS spam detection practical was implemented using Apache Spark and PySpark. The workflow processes the SMS dataset, performs tokenization and stop-word removal, generates TF-IDF features and trains Logistic Regression models using Spark-based distributed computation.

Hyperparameter tuning is performed for `regParam`, `maxIter` and `elasticNetParam`. The trained model is evaluated using Accuracy, Precision, Recall, F1-Score and ROC-AUC. The practical also compares full-batch and mini-batch stochastic gradient descent and records the effect of dataset size on training time and accuracy.

The exact numerical performance values are produced by `practical11.py` at execution time and should be captured from the VS Code terminal rather than being manually invented in the report.

Key Questions / Analysis

Q1. What is the difference between full-batch, mini-batch, and stochastic gradient descent? Why is mini-batch preferred in distributed settings?

Full-batch gradient descent uses the complete training dataset for each parameter update. Stochastic gradient descent uses one training example at a time. Mini-batch gradient descent uses a small subset of records for each update. Mini-batches provide a practical balance between computational efficiency and stable updates and are suitable for distributed data processing.

Q2. How does the TF-IDF vectorizer handle the 'curse of dimensionality' for text classification?

TF-IDF converts text into numerical feature vectors while assigning greater importance to terms that help distinguish documents. In this practical, `HashingTF` fixes the feature-space size before IDF weighting, keeping the representation bounded and suitable for large-scale processing.

Q3. What is L1 vs L2 regularization? How does `elasticNetParam` blend them in Spark MLlib?

L1 regularization encourages sparse coefficients, while L2 regularization penalizes large coefficients. `elasticNetParam` controls the mixture of L1 and L2 regularization. A value of 0 corresponds to L2 regularization, while higher values introduce more L1 contribution.

Q4. In a distributed environment, how do gradient updates from multiple workers get aggregated? What is the risk of stale gradients?

Spark divides the data into partitions and distributes computation across workers or CPU cores. Distributed optimization combines the information needed for parameter updates. In asynchronous optimization, stale gradients may be calculated from older parameter values and can slow convergence or reduce optimization stability.

Q5. If Recall is more important than Precision for spam detection, how would you adjust the classification threshold?

A lower classification threshold generally causes more messages to be classified as spam. This can increase recall and reduce missed spam, but it may also increase false positives and lower precision. The threshold should therefore be selected according to the relative cost of missed spam and incorrectly flagged legitimate messages.

Supplementary Problems

- Apply HashingTF instead of TF-IDF and compare model performance and feature-space size.
- Increase the training data from 500K toward 5M records and plot the learning curve.
- Implement a Spark ML Pipeline using Tokenizer -> StopWordsRemover -> HashingTF -> IDF -> LogisticRegression.

Tools / Technology used

Apache Spark 3.x with PySpark | Spark MLlib (LogisticRegression, LogisticRegressionWithSGD) | Spark ML (RegexTokenizer, StopWordsRemover, HashingTF, IDF) | VS Code | Python 3.11 | CSV

Conclusion

Apache Spark and PySpark successfully provide a scalable environment for large-scale SMS spam classification. The practical demonstrates the complete text-classification workflow, including dataset loading, text preprocessing, TF-IDF feature extraction, Logistic Regression, distributed stochastic gradient descent and model evaluation.

The practical also demonstrates hyperparameter tuning and compares full-batch and mini-batch training. Dataset-size experiments provide a way to observe how increasing the amount of training data affects training time and classification accuracy.

The solution is configured for VS Code using Spark local[*] mode, so it can be executed without HDFS on a suitable student computer. The practical specification describes HDFS for a large-scale deployment, while local Spark mode is used here for convenient practical execution.

Github Link

https://github.com/RudraMaiyariya/D25DIT081_BDA_SHP_2026/tree/main/Practical11

